Combining Observable JS and Shiny in a Single Quarto Document

A practical guide to mixing client-side and server-side reactivity

Ronald 'Ryy' G. Thomas

2025-12-01

Table of contents

1 Introduction **2**
- 1.1 Motivations . . . 3
- 1.2 Objectives . . . 3

2 Prerequisites and Setup **3**

3 What is Client-Side versus Server-Side Reactivity? **4**

4 Getting Started **4**
- 4.1 The Side-by-Side Comparison . . . 4
  - 4.1.1 Observable JS . . . 4
  - 4.1.2 Shiny . . . 5
- 4.2 Key Syntax Differences . . . 6

5 The Challenges **8**
- 5.1 Challenge 1: FileAttachment Does Not Work . . . 8
- 5.2 Challenge 2: ojs_define Fails Too . . . 8
- 5.3 Challenge 3: Local d3.csv Returns 404 . . . 8
- 5.4 The Solution: Fetch from a Public URL . . . 9

6 Dos and Don'ts **9**
- 6.1 Additional Gotchas . . . 9
- 6.2 Things to Watch Out For . . . 10
- 6.3 Lessons Learnt . . . 10
  - 6.3.1 Conceptual Understanding . . . 10
  - 6.3.2 Technical Skills . . . 11
  - 6.3.3 Gotchas and Pitfalls . . . 11
- 6.4 Limitations . . . 11
- 6.5 Opportunities for Improvement . . . 12

7 Wrapping Up **12**

8 See Also 12

9 Reproducibility 13

10 Let's Connect 13

Figure 1: A split screen showing two reactive frameworks working side by side in a single document

Combining two reactive frameworks in one document is harder than it looks.

1 Introduction

I did not really know how difficult it would be to combine Observable JS and Shiny in a single Quarto document until I tried to create a side-by-side comparison. Like many statisticians, I assumed that since Quarto supports both frameworks, mixing them would be straightforward. Thirty minutes of setup turned into several hours of debugging.

The core difficulty is that Observable JS and Shiny use fundamentally different execution models. Observable runs entirely in the browser; Shiny requires a server process. When you add `server: shiny` to a Quarto document, the rendering context changes in ways that break standard Observable data-loading patterns.

This post documents the journey from naive optimism to a working solution. Every data-loading approach I tried failed – `FileAttachment`, `ojs_define`, local `d3.csv` – until I discovered that fetching from a public URL bypasses the conflict entirely.

1.1 Motivations

- I wanted to compare client-side and server-side reactive frameworks in a single document for a workshop demonstration.
- Colleagues frequently ask which framework to use for interactive visualisations, and a side-by-side comparison seemed more persuasive than a verbal explanation.
- The Quarto documentation implies that OJS and Shiny can coexist, but provides little guidance on the practical difficulties of combining them.
- I was curious whether the multi-language support in Quarto truly extends to mixing reactive systems, or whether it is limited to sequential code blocks.
- I needed to document the gotchas for my own future reference and for others attempting the same combination.

1.2 Objectives

1. Create identical interactive visualisations using both Observable JS and Shiny, displayed side-by-side in a single Quarto document.
2. Document the data-loading approaches that fail (`FileAttachment`, `ojs_define`, `d3.csv`) and explain why they fail in the Shiny context.
3. Provide a working solution using `fetch()` from a public URL that bypasses the server-client conflict.
4. Summarise the key syntax differences between OJS and Shiny for statisticians evaluating both frameworks.

I am documenting my learning process here. If you spot errors or have better approaches, please let me know.

Understanding the architectural differences between client-side and server-side reactivity.

2 Prerequisites and Setup

To follow along with this post, you will need:

- Quarto (1.3 or later)
- R with ggplot2 and dplyr packages
- A web browser (Chrome or Firefox recommended)

The Observable JS code runs in the browser and requires no additional installation. The Shiny code requires a running R server, which Quarto manages automatically when `server: shiny` is set.

The Palmer Penguins dataset is used for both frameworks. For the OJS side, data is fetched from a public URL. For the Shiny side, data is read from a local CSV file.

3 What is Client-Side versus Server-Side Reactivity?

Client-side reactivity means that all computation happens in the browser. Observable JS exemplifies this approach: data is loaded into the browser, and user interactions trigger JavaScript functions that filter, transform, and visualise data without contacting a server. The advantage is speed; the limitation is that the browser must hold all data in memory.

Server-side reactivity means that user interactions trigger requests to a server process, which performs computation and returns results to the browser. Shiny exemplifies this approach: R runs on the server, and the browser displays rendered output. The advantage is access to the full R ecosystem and arbitrarily large datasets; the limitation is network latency and server resource requirements.

When you combine both in a single Quarto document, you are running a Shiny server that also serves Observable JS. This dual context is where the data-loading conflicts arise.

4 Getting Started

4.1 The Side-by-Side Comparison

Below are two identical visualisations – one built with Observable JS (client-side) and one with Shiny (server-side). Both filter the Palmer Penguins dataset by bill length and display a histogram of body mass.

4.1.1 Observable JS

```
//| echo: true

viewof bill_min_ojs = Inputs.range(
  [32, 50],
  {value: 35, step: 1,
   label: "Min bill length (mm):"}
)

penguins_raw = {
  const response = await fetch(
    "https://raw.githubusercontent.com/" +
    "allisonhorst/palmerpenguins/" +
    "main/inst/extdata/penguins.csv"
  );
  const text = await response.text();
  return d3.csvParse(text, d3.autoType);
}

data_ojs = penguins_raw.filter(
```

```
  d => d.bill_length_mm != null &&
       d.body_mass_g != null
)

filtered_ojs = data_ojs.filter(
  d => d.bill_length_mm > bill_min_ojs
)

Plot.plot({
  height: 250,
  marks: [
    Plot.rectY(
      filtered_ojs,
      Plot.binX(
        {y: "count"},
        {x: "body_mass_g", fill: "species"}
      )
    ),
    Plot.ruleY([0])
  ]
})
```

Count: ${filtered_ojs.length} penguins

4.1.2 Shiny

```
sliderInput(
  inputId = "bill_min_shiny",
  label = "Min bill length (mm):",
  min = 32, max = 50, value = 35, step = 1
)

plotOutput("hist_shiny", height = "250px")
textOutput("count_shiny")

library(ggplot2)
library(dplyr)

data_shiny <- read.csv(
  "data/raw_data/palmer-penguins.csv"
)

filtered_shiny <- reactive({
  data_shiny |>
    filter(
```

```r
      bill_length_mm > input$bill_min_shiny
    ) |>
    filter(
      !is.na(body_mass_g),
      !is.na(species)
    )
})

output$hist_shiny <- renderPlot({
  ggplot(
    filtered_shiny(),
    aes(x = body_mass_g, fill = species)
  ) +
    geom_histogram(bins = 20) +
    theme_minimal() +
    labs(x = "Body Mass (g)", y = "Count")
})

output$count_shiny <- renderText({
  paste(
    "Count:",
    nrow(filtered_shiny()),
    "penguins"
  )
})
```

4.2 Key Syntax Differences

Aspect	Observable JS	Shiny
Input	`viewof x = Inputs.range()`	`sliderInput("x", ...)`
Access value	Use `x` directly	`input$x`
Data loading	`fetch()`	`read.csv()`
Filtering	`data.filter()`	`reactive({})`
Plot	`Plot.plot({...})`	`renderPlot({...})`
Reactivity	Implicit	Explicit wrappers

Figure 2: A debugging session with multiple terminal windows open and documentation visible

Debugging reactive framework conflicts requires patience and systematic elimination.

5 The Challenges

Getting the side-by-side comparison working was surprisingly difficult. Each approach I tried revealed a different aspect of the server-client conflict.

5.1 Challenge 1: FileAttachment Does Not Work

My first attempt used Observable's standard data-loading mechanism:

```
// THIS DOES NOT WORK IN SHINY DOCUMENTS
data = FileAttachment(
  "palmer-penguins.csv"
).csv({typed: true})
```

Result: `OJS Error: Unable to load file`

Why: When you add `server: shiny`, the document runs as a Shiny application. The Shiny server does not know about Observable's file attachment system. The file resolution mechanism that works in static Quarto documents is unavailable in the Shiny context.

5.2 Challenge 2: ojs_define Fails Too

The Quarto documentation mentions `ojs_define()` for passing data from R to OJS:

```
#| context: server
data <- read.csv("penguins.csv")
ojs_define(my_data = data)
```

Result: `Error: Unexpected data.frame object... Did you forget to use a render function?`

Why: In the Shiny context, `ojs_define()` expects reactive expressions, not static data frames. The behaviour differs from what the documentation shows for regular (non-Shiny) Quarto documents.

5.3 Challenge 3: Local d3.csv Returns 404

I tried D3's CSV loader as an alternative:

```
// THIS ALSO DOES NOT WORK
data = d3.csv(
  "palmer-penguins.csv", d3.autoType
)
```

Result: `OJS Error: 404 Not Found`

Why: The Shiny server does not serve local static files in the way that Observable expects. The file is present on disk but not accessible through the URL that `d3.csv` constructs.

5.4 The Solution: Fetch from a Public URL

The only reliable approach I found is fetching data from a public URL:

```
penguins_raw = {
  const response = await fetch(
    "https://raw.githubusercontent.com/" +
    "allisonhorst/palmerpenguins/" +
    "main/inst/extdata/penguins.csv"
  );
  const text = await response.text();
  return d3.csvParse(text, d3.autoType);
}
```

This bypasses the Shiny server entirely by making an HTTP request to an external source. The browser handles the fetch independently of the server process.

6 Dos and Don'ts

Do:

1. Use `fetch()` with public URLs for OJS data in Shiny documents.
2. Keep OJS and Shiny data loading completely separate.
3. Test incrementally: get Shiny working first, then add OJS.
4. Hard-refresh your browser (Cmd+Shift+R) when debugging.
5. Restart the preview server after major changes.

Do not:

1. Use `FileAttachment()` in Shiny documents.
2. Use `ojs_define()` with static data in the Shiny context.
3. Assume `d3.csv("local.csv")` will work.
4. Trust displayed code during debugging; browser caching causes confusion.
5. Mix OJS and Shiny logic in the same code block.

6.1 Additional Gotchas

Browser caching: The browser often shows old code during debugging. The displayed code might show `FileAttachment(...)` even though your file now uses `fetch()`.

Solution: Kill the server, delete generated files, restart, and hard-refresh:

```
rm -rf yourfile_files yourfile.html
quarto preview yourfile.qmd
```

Port changes: Every restart may assign a different port (5155, 6532, 7665). Check the terminal output for the correct URL.

Misleading error messages: The error "Did you forget to use a render function?" does not mean you need `renderSomething()`. It means `ojs_define()` does not work with static data in the Shiny context.

6.2 Things to Watch Out For

1. **FileAttachment silently fails.** There is no helpful error message explaining that the Shiny context disables Observable's file system. You see a generic load error.
2. **Browser caching masks your fixes.** After changing from `FileAttachment` to `fetch`, the browser may still display the old code and the old error. Hard-refresh every time.
3. **Port numbers change on restart.** Do not bookmark a specific port; check the terminal output after each `quarto preview`.
4. **OJS and Shiny cannot share data.** There is no reliable way to pass data between the two contexts. Keep them independent.
5. **The Quarto documentation does not cover this edge case well.** Expect to rely on experimentation rather than official guidance.

Figure 3: A whiteboard with architectural diagrams comparing client-server models

Understanding the architectural boundary between client-side and server-side is the key insight.

6.3 Lessons Learnt

6.3.1 Conceptual Understanding

- Observable JS and Shiny use fundamentally different execution models: client-side versus server-side. Combining them means running both simultaneously.

- The Shiny server context overrides Observable's file resolution system. File loading that works in static Quarto documents breaks in Shiny documents.
- Data isolation between the two frameworks is not a limitation to work around but a constraint to accept. Attempting to share data between contexts leads to fragile, unreliable solutions.
- The `fetch()` API provides a clean escape hatch because it operates at the HTTP level, independent of both frameworks' internal resolution mechanisms.

6.3.2 Technical Skills

- Using `fetch()` with `d3.csvParse()` for OJS data loading in Shiny documents became the reliable pattern.
- Keeping OJS and Shiny code in strictly separate blocks prevents context contamination.
- Hard-refreshing the browser (Cmd+Shift+R) and deleting generated files became essential debugging habits.
- Understanding the `#| context: server` chunk option clarified how Shiny server code differs from UI code in Quarto.

6.3.3 Gotchas and Pitfalls

- `FileAttachment()` does not work in Shiny documents, and the error message does not explain why.
- `ojs_define()` behaves differently depending on whether the document uses `server: shiny` or not.
- Browser caching during debugging causes the most time-consuming confusion.
- Error messages from OJS in the Shiny context are often misleading; they reference the wrong cause.

6.4 Limitations

- This approach requires hosting data at a public URL for the OJS side, which may not be acceptable for sensitive or proprietary datasets.
- There is no reliable data sharing between OJS and Shiny contexts; each framework loads and processes data independently.
- Debugging is significantly harder than in single-framework documents because two reactive systems interact through the browser.
- The Quarto documentation does not cover this combination in detail, making experimentation the primary learning method.
- Performance may suffer because the browser runs OJS reactivity while simultaneously communicating with the Shiny server.
- The `fetch()` workaround depends on network availability; offline use is not supported for the OJS side.

6.5 Opportunities for Improvement

1. Test `ojs_define()` with `reactive()` wrappers in the Shiny server context to determine whether reactive expressions can bridge the gap.
2. Create a helper function or Quarto extension that transparently handles data loading for both frameworks.
3. Explore Shiny's resource-serving configuration to determine whether local files can be made available to OJS through custom routes.
4. Document a clean workflow for teams that need both frameworks in production, including testing strategies and debugging protocols.
5. Investigate whether Quarto's OJS runtime could be modified to respect Shiny's static file serving mechanism.
6. Build a minimal reproducible example repository that demonstrates the working pattern for others to fork.

7 Wrapping Up

Combining Observable JS and Shiny in a single Quarto document is possible but not straightforward. The fundamental issue is that the Shiny server context disables Observable's standard file-loading mechanisms, and the two frameworks cannot reliably share data.

What I learnt most from this exploration is that the boundary between client-side and server-side execution is not merely architectural – it is a hard constraint that determines which approaches work and which fail silently. Once I accepted that the two frameworks must remain independent, the `fetch()` solution became obvious.

For educational purposes – comparing both technologies side-by-side – this approach is valuable once you get past the setup hurdles. For production applications, I recommend choosing one framework rather than combining both.

Main takeaways:

- `FileAttachment()`, `ojs_define()`, and `d3.csv()` all fail in the Shiny context.
- `fetch()` from a public URL is the reliable workaround.
- Keep OJS and Shiny data loading completely independent.
- Hard-refresh your browser after every change when debugging.

8 See Also

Related posts:

- Prototyping a Shiny App with ChatGPT – Building a Shiny app iteratively with AI assistance

Key resources:

- Quarto OJS Documentation – Official Observable JS integration guide
- Quarto Shiny Documentation – Official Shiny integration guide
- Observable Plot – The plotting library used in the OJS examples

- Palmer Penguins Dataset – The dataset used in both frameworks
- D3.js: Data-Driven Documents – The JavaScript library underlying Observable Plot

9 Reproducibility

This post contains live Observable JS code that runs in the browser and Shiny server code that is displayed but not evaluated (set to `eval: false` for static rendering). To run the full interactive version:

```
cd ~/prj/qblog/posts/34-shinyvsobservable/
cd shinyvsobservable
quarto preview analysis/report/index.qmd
```

The OJS code fetches data from a public GitHub URL and requires network access. The Shiny code reads from a local CSV file at `data/raw_data/palmer-penguins.csv`.

Project files:

```
shinyvsobservable/
  analysis/report/index.qmd  -- This post
  data/raw_data/              -- Penguins CSV
  analysis/media/images/      -- Hero, ambiance
```

10 Let's Connect

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas
- **Email:** rgtlab.org/contact

I would enjoy hearing from you if:

- You spot an error or a better approach to any of the code in this post.
- You have suggestions for topics you would like to see covered.
- You want to discuss R programming, data science, or reproducible research.
- You have questions about anything in this tutorial.
- You just want to say hello and connect.